

Exploring Hawaii

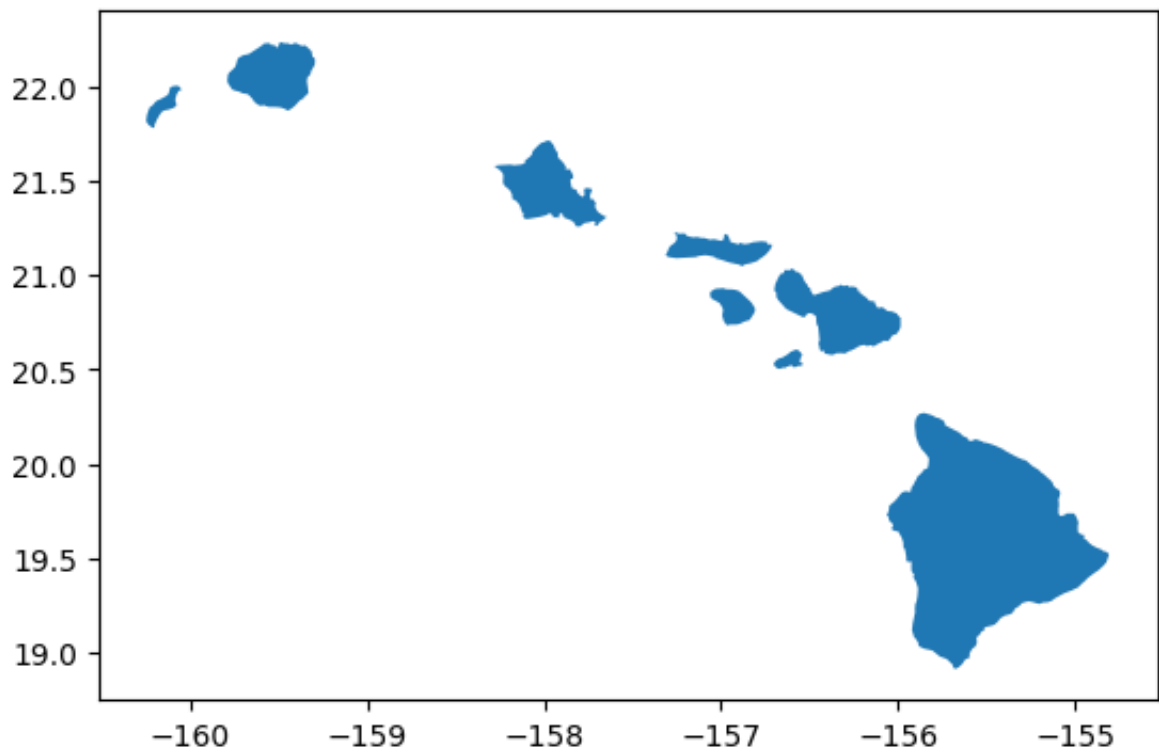
Geographic data (boundaries) for the Hawaiian islands can be found in the standard file for the 50 states plus DC and Puerto Rico.

```
In [1... import os
import geopandas as gpd
import matplotlib.pyplot as plt
import shapely

user = os.path.expanduser("~")
dbpath = user + '/data/'
fn = 'gz_2010_us_040_00_5m'
gdf = gpd.read_file(dbpath+fn)
gdf.shape
```

Out[1... (52, 6)

```
In [2... sel = gdf['NAME'] == 'Hawaii'
hawaii = gdf[sel]
hawaii.plot()
plt.show()
```



```
In [3... hawaii.shape
```

```
Out[3... (1, 6)
```

So there is only one row of data for Hawaii, but 9 islands. There may be more sophisticated ways to do this, but one approach is as follows.

`hawaii.geometry` is a GeoSeries. We can index into it with `.iloc`.

```
In [4... MP = hawaii.geometry.iloc[0]
```

`0` is the only valid index into `hawaii.geometry`. `MP` is a `shapely MULTIPOLYGON`.

To get the components of `MP` we get the geometries of it. These are going to be `POLYGON` objects.

```
In [5... type(MP)
```

```
Out[5... shapely.geometry.multipolygon.MultiPolygon
```

```
In [6... pg = MP.geoms[0]  
print(type(pg))  
pg
```

```
<class 'shapely.geometry.polygon.Polygon'>
```

```
Out[6...
```



The first one is clearly the big island, Hawaii. I couldn't find the `geoms` method in the `shapely` docs. I did find this, which does the same thing.

```
In [7... shapely.get_geometry(MP, 0)
```

```
Out[7...
```



```
In [8... MP.geoms
```

```
Out[8... <shapely.geometry.base.GeometrySequence at 0x10d4e1f90>
```

```
In [9... islands_mp = list(MP.geoms)
len(islands_mp)
```

```
Out[9... 9
```

```
In [1... islands_mp
```

```
Out[1... [

```

`islands_mp` is an array (`list`) of POLYGON objects. The names of the islands are not given in the data. We need to decide which is which.

One way to figure them out would be to obtain points from Google Maps. I just picked a random point in the interior of each, and then reversed LAT,LON to the Geopandas convention of LON,LAT (x,y).

```
In [1... D = {"Hawaii": [-155.519783, 19.625055],
      "Kaho'olawe": [-156.607857, 20.550829],
      "Kauai": [-159.567160, 22.017814],
      "Lanai": [-156.930387, 20.834303],
      "Maui": [-156.279557, 20.758340],
      "Molokai": [-156.986996, 21.134644],
      "Ni'ihau": [-160.148047, 21.904692],
```

```
"O'ahu": [-157.968125, 21.488976],
"Ford Island": [-157.959627, 21.363596]}
```

```
In [1... from shapely import Point

rL = list()
for i,pg in enumerate(islands_mp):
    for k,v in D.items():
        if pg.contains(Point(v)):
            rL.append((i,k))

for t in rL:
    print(t)
```

```
(0, 'Hawaii')
(2, 'Ni'ihau')
(3, 'Kauai')
(4, 'Molokai')
(5, 'Kaho'olawe')
(6, 'Maui')
(7, 'Lanai')
(8, 'O'ahu')
(8, 'Ford Island')
```

That looks good, except there is an issue with Ford Island. That's not too surprising, since Ford is a small island in the harbor at Honolulu on Oahu. What I'm guessing is that 1 is Ford Island, but the point I picked isn't quite right, or something.

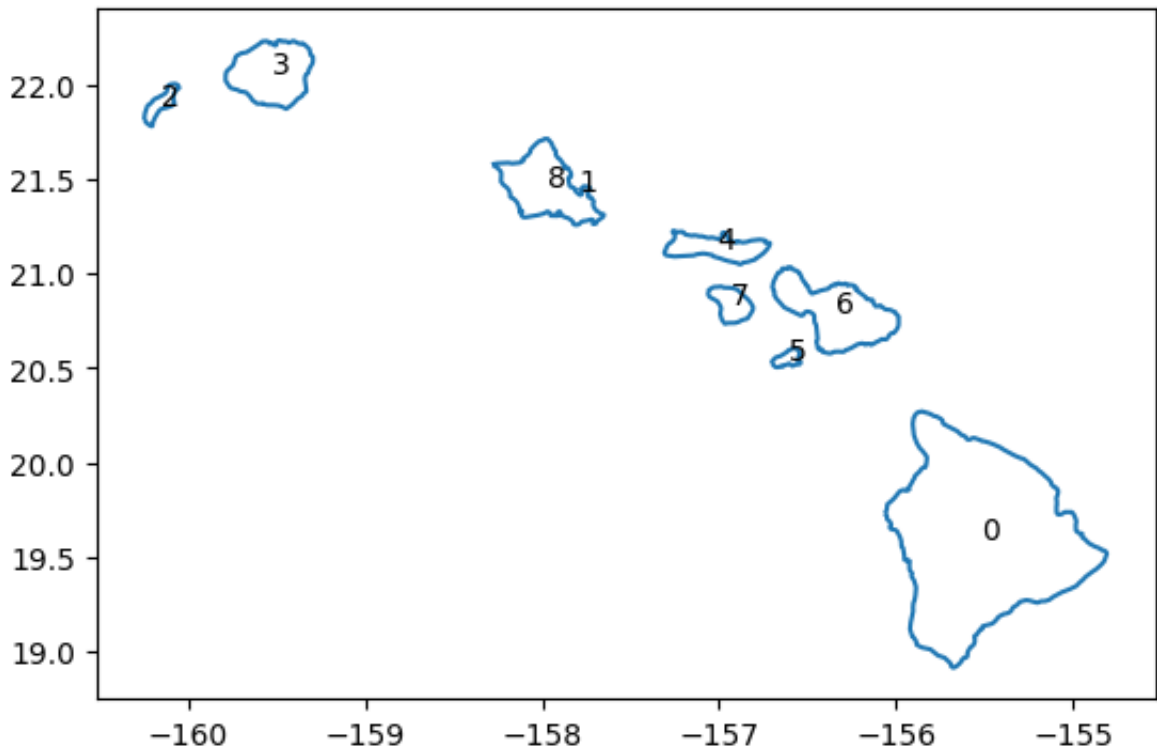
We can also get at the list of polygons by `.explode`.

```
In [1... gs = hawaii.geometry # gs is a GeoSeries
islands_gs = gs.explode(index_parts=True) # still a GeoSe

L = list(range(9)) # trick to get an index inside apply
def f(p):
    #x,y = p.representative_point().coords[:,0]
    x = p.centroid.x
    y = p.centroid.y
    index = L.pop(0)
    plt.annotate(
        text=str(index),
        xy=(x,y))
```

We use `apply` to feed the rows of `islands_gs` to the function `f`.

```
In [1... fig,ax = plt.subplots()
islands_gs.boundary.plot(ax=ax)
islands_gs.apply(f)
plt.show()
```



We will make the entries for a column of island names by manually massaging `rL` from above. The question is whether it is Ford Island or O'ahu that should be item number 1.

I wanted to use `.area()` to decide this question, but GeoPandas gave me an error about CRS, even after I do `islands_gdf = islands_gdf.to_crs(hawaii.crs)`. Later, I figured out the reason: you must project to a crs that is appropriate for area.

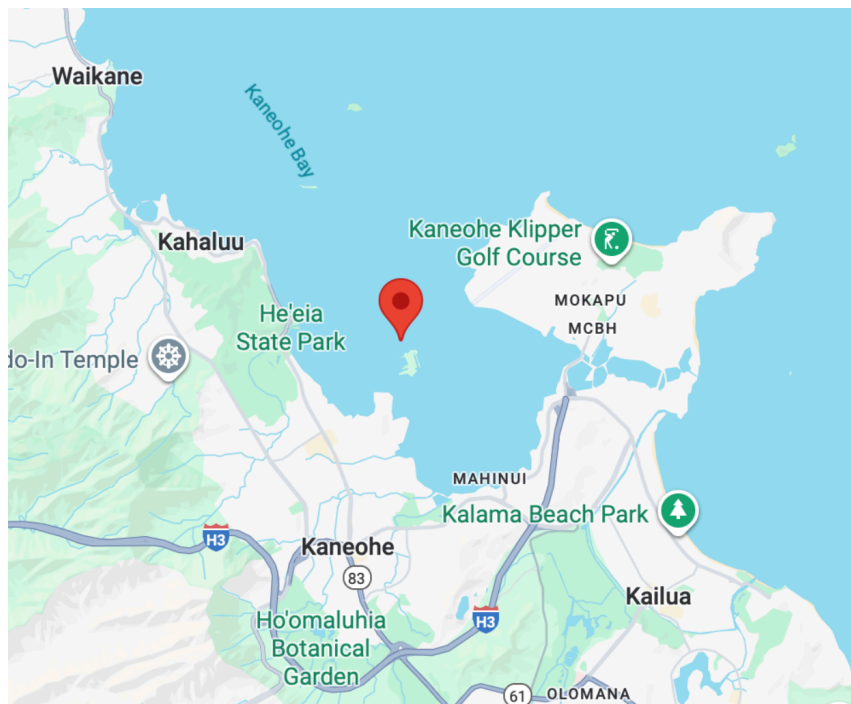
```
In [1... crs = islands_gs.estimate_utm_crs()
mod = islands_gs.to_crs(crs)
mod.area
```

```
Out [1... 11 0 1.046297e+10
          1 1.281197e+05
          2 1.863389e+08
          3 1.440377e+09
          4 6.733605e+08
          5 1.149144e+08
          6 1.887029e+09
          7 3.637272e+08
          8 1.572581e+09
dtype: float64
```

Clearly, 8 is much larger than 1. So 1 must be Ford Island.

Another idea was to just search with the first point of polygon 1 on Google Maps (remembering to reverse and put latitude first). The result lies off the north shore.

That suggests I should assign polygon 1 to Oahu. Hmm. That is an island but it's not Ford.

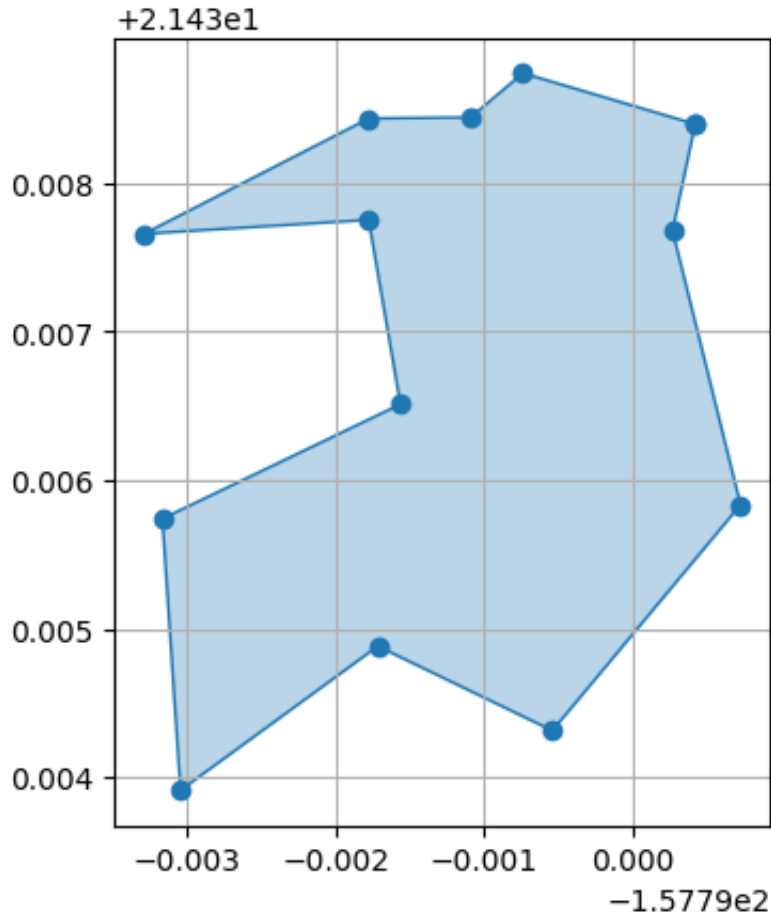


```
In [1... names = [t[1] for t in rL]
ford = names.pop()
names.insert(1, ford)
names
```

```
Out[1...] ['Hawaii',
            'Ford Island',
            "Ni'ihau",
            'Kauai',
            'Molokai',
            "Kaho'olawe",
            'Maui',
            'Lanai',
            "O'ahu"]
```

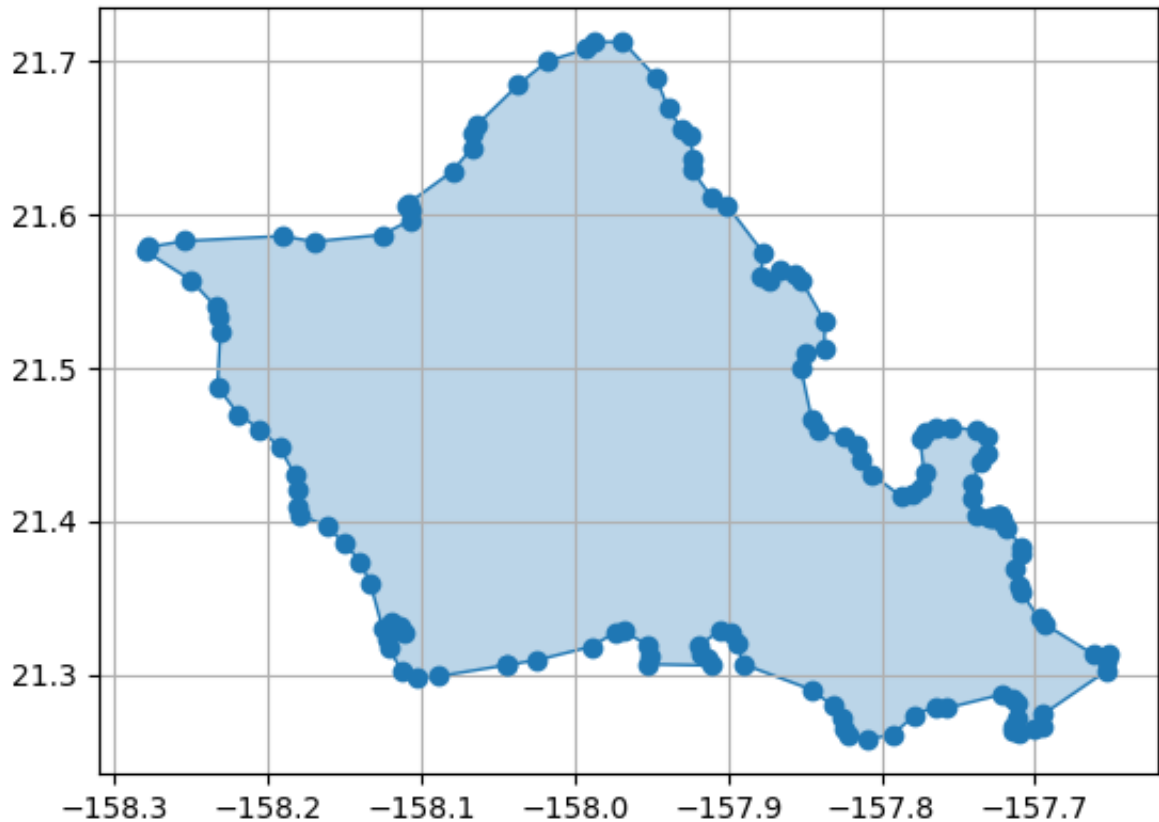
```
In [1...] pg = islands_gs.iloc[1]
            from shapely import plotting
            plotting.plot_polygon(pg)
```

```
Out[1...] (<matplotlib.patches.PathPatch at 0x10d6add10>,
            <matplotlib.lines.Line2D at 0x10d6adf90>)
```



```
In [1...] pg = islands_gs.iloc[8]
            from shapely import plotting
            plotting.plot_polygon(pg)
```

Out[1... (<matplotlib.patches.PathPatch at 0x10d71e5d0>,
<matplotlib.lines.Line2D at 0x10d71e710>)



I guess that settles it. Clearly #8 is O'ahu.

Annotating in the "proper" way would involve making a column.

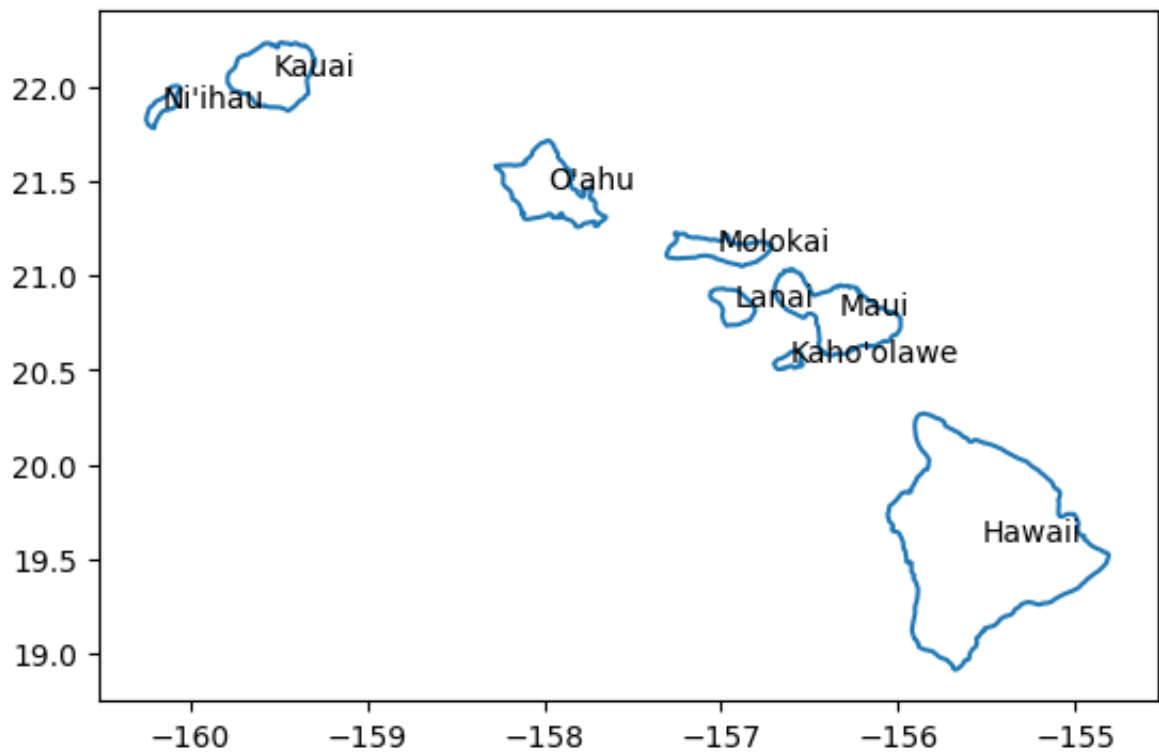
GeoPandas is complaining about the crs even though `islands_gs` has one and even if I set it again.

So, let's just reuse the code from above.

```
In [1... L = list(range(9))

def f(p):
    #x,y = p.representative_point().coords[:,0]
    x = p.centroid.x
    y = p.centroid.y
    index = L.pop(0)
    if index != 1:
        plt.annotate(
            text=names[index],
            xy=(x,y))
```

```
In [2... fig,ax = plt.subplots()  
islands_gs.boundary.plot(ax=ax)  
islands_gs.apply(f)  
plt.show()
```

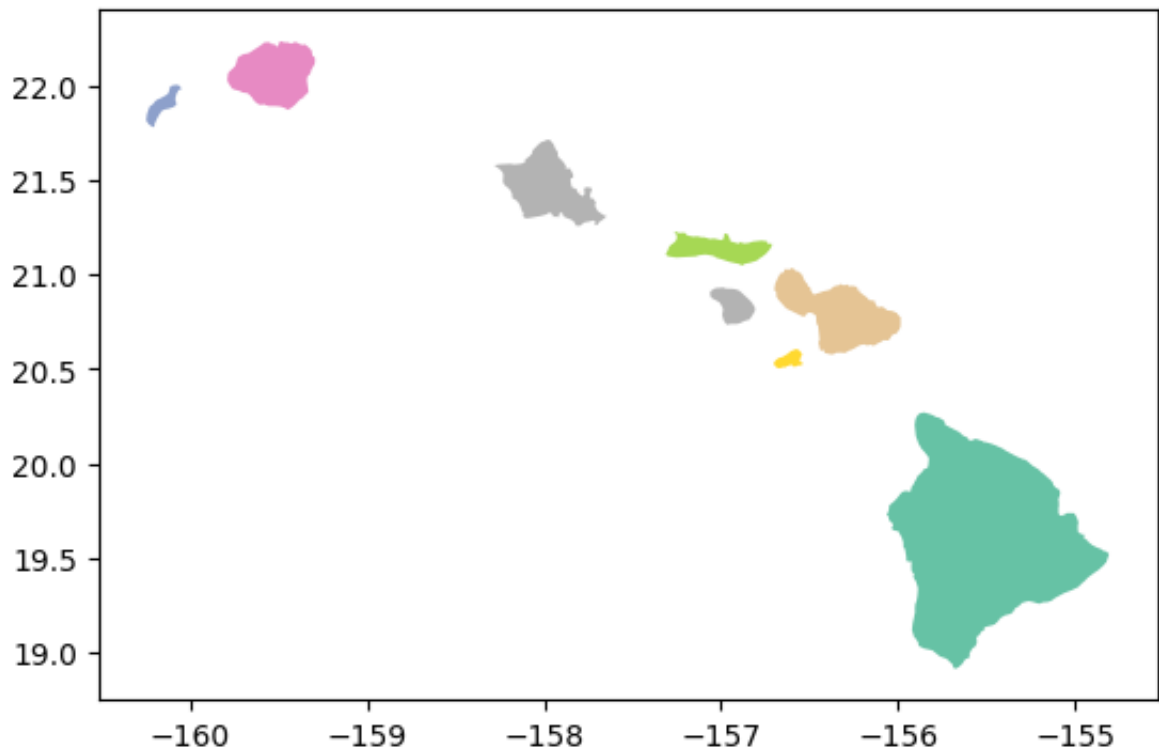


The fussy part is then to adjust positions for the names. You can find that in a script in the Hawaii folder. Just adjust `x` and `y` inside `f` based on the name.

A much simpler way is to add the labels in another application.

One last thing is to get some nice colors.

```
In [2... islands_gs.plot(cmap='Set2')  
plt.show()
```



Let's just take a last look at one of the islands: Maui.

```
In [2... maui = islands_gs.iloc[6]
maui
```



```
In [2... type(maui)
```

```
Out[2... shapely.geometry.polygon.Polygon
```

```
In [2... maui_gs = gpd.GeoSeries(maui)
```

```
In [2... D = {}
maui_gdf = gpd.GeoDataFrame(D, geometry=maui_gs, crs=hawaii
maui_gdf
```

```
Out[2... geometry
0 POLYGON ((-156.61201 21.02477, -156.61206 21.0...
```

```
In [2... t = maui.representative_point().coords[:]
x = t[0][0]
y = t[0][1]
type(x)
```

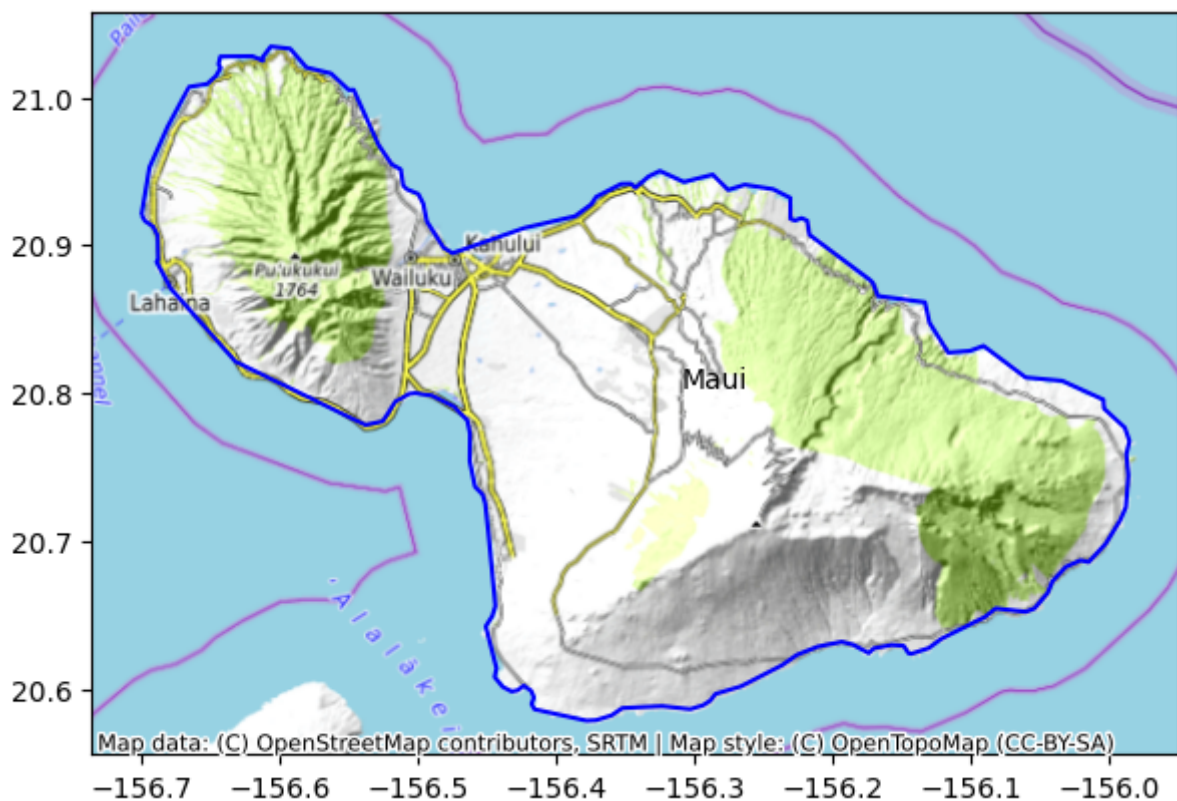
Out[2... float

This time we'll add a basemap.

```
In [2... import contextily as cx

ax = maui_gdf.boundary.plot(color='b',figsize=(7, 7))
cx.add_basemap(ax,source=cx.providers.OpenTopoMap,
               crs=maui_gdf.crs)

plt.annotate(text='Maui',xy=(x,y))
plt.show()
```

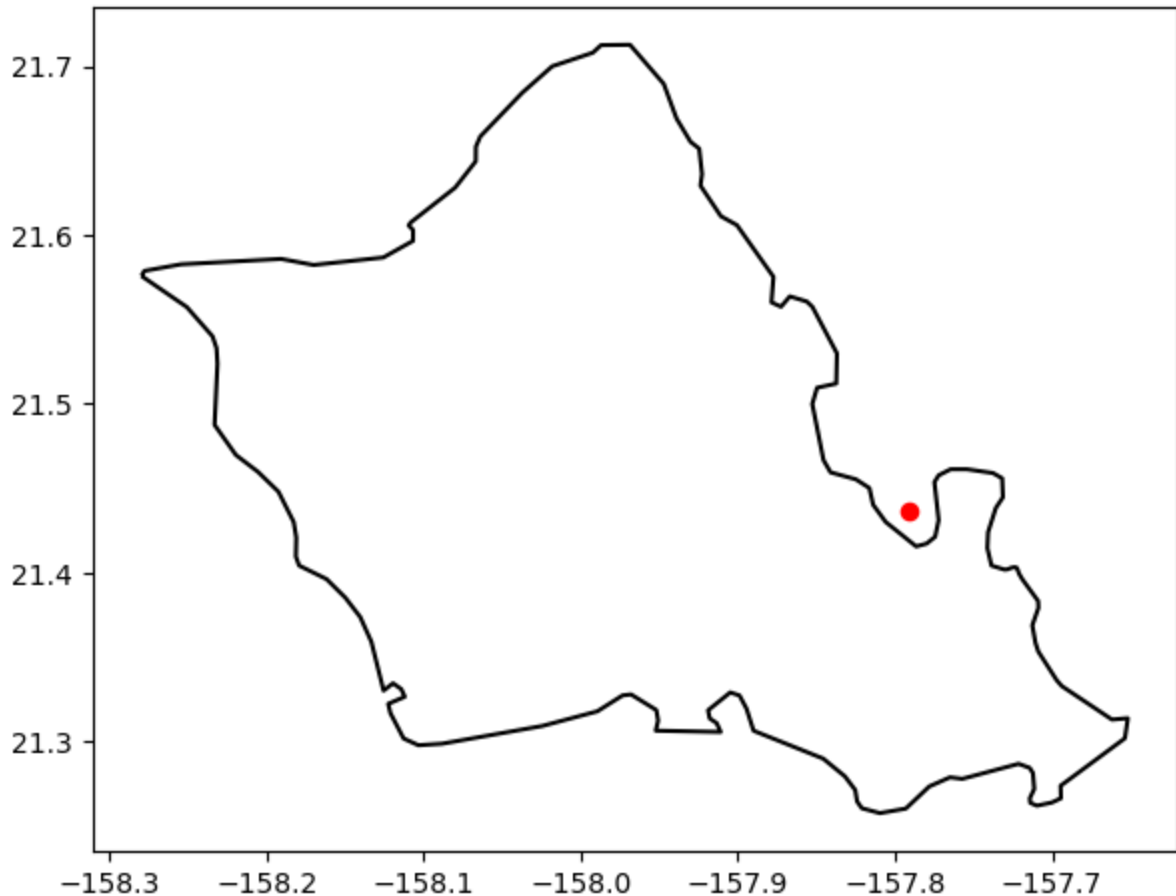


```
In [2... oahu = islands_gs.iloc[8]
oahu_gs = gpd.GeoSeries(oahu)
oahu_gdf = gpd.GeoDataFrame(D,geometry=oahu_gs,crs=hawaii

ford = islands_gs.iloc[1]
cent = ford.centroid
```

```
x = cent.x
y = cent.y

ax = oahu_gdf.boundary.plot(color='k',figsize=(7, 7))
ax.scatter([x],[y],s=35,color='r')
plt.show()
```

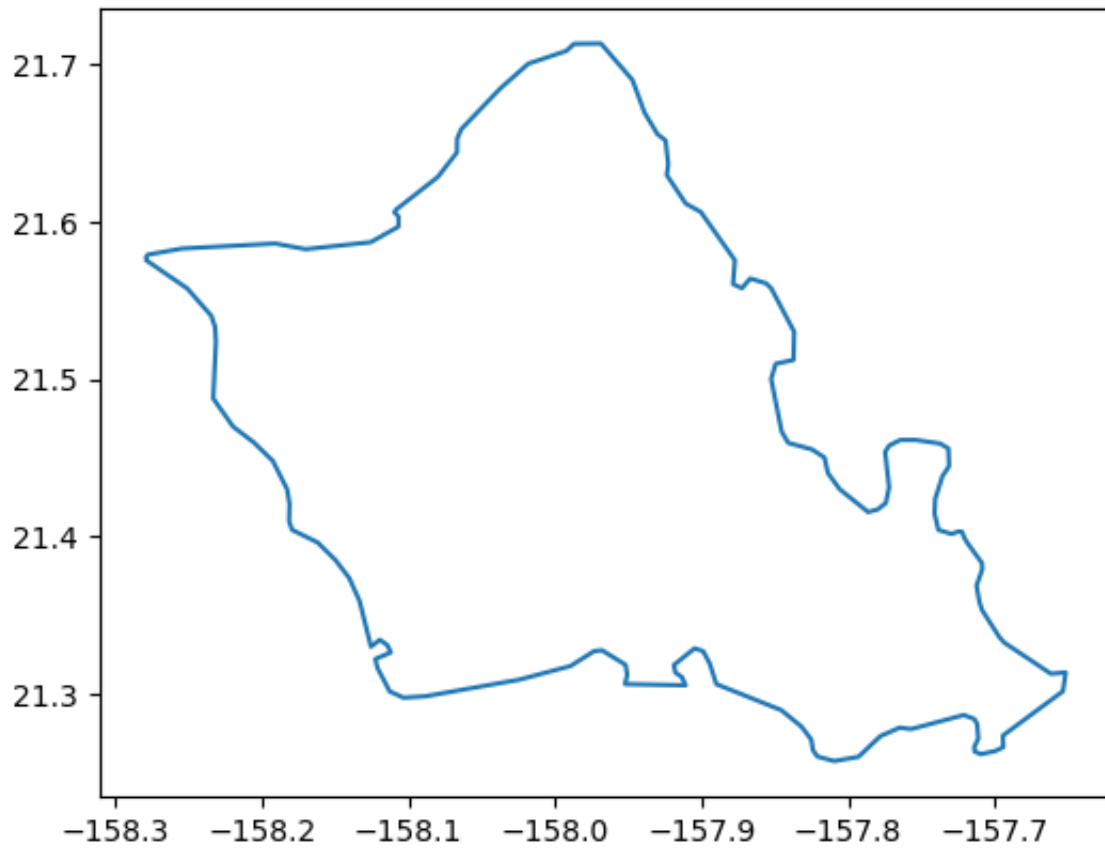


It's becoming clear. That's not Ford Island but Moko-ule Island (Coconut Island) in Kaneohe Bay on the north shore. Why it's in this data set as a separate place isn't clear to me.

```
In [2...] type(oahu)
```

```
Out[2...] shapely.geometry.polygon.Polygon
```

```
In [3...] #gdf.cx[xmin:xmax, ymin:ymax]
bay = oahu_gdf.cx[-158:-157.9,21.35:21.38]
bay.boundary.plot()
plt.show()
```



This application of `.cx` isn't working. I'm not sure why that is.